Travaux Pratiques
Algorithmes de Clustering

4ème Année Ingénieur - Machine Learning

Durée : 3 heures
Prérequis : Python, NumPy, Scikit-learn, Matplotlib, OpenCV
Objectif général : Maîtriser les algorithmes de clustering non supervisé

I. Introduction

Objectifs pédagogiques :

- Comprendre les principes du clustering non supervisé
- Implémenter et comparer différents algorithmes de clustering
- Évaluer la qualité des clusterings avec des métriques appropriées

1.1 Présentation des algorithmes

- K-means; DBSCAN; mean-shift; GMM; CAH

1.2 Métriques d'évaluation

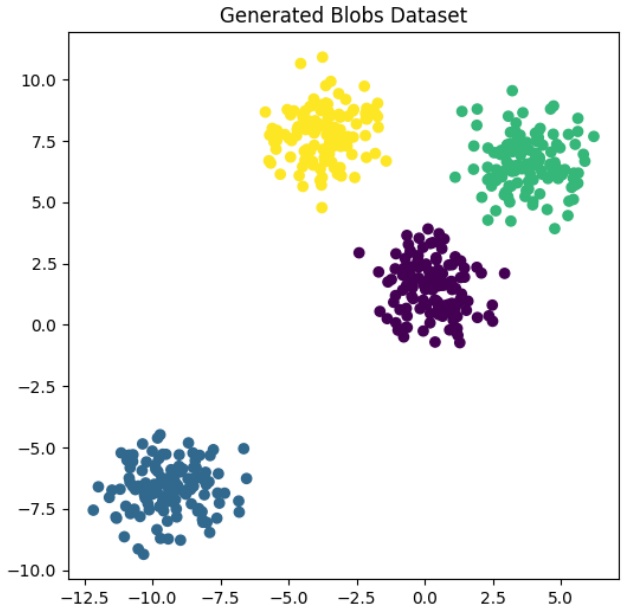
- **Silhouette Score :** Mesure la cohésion et la séparation des clusters
- **Adjusted Rand Index (ARS) :** Compare avec les vraies étiquettes

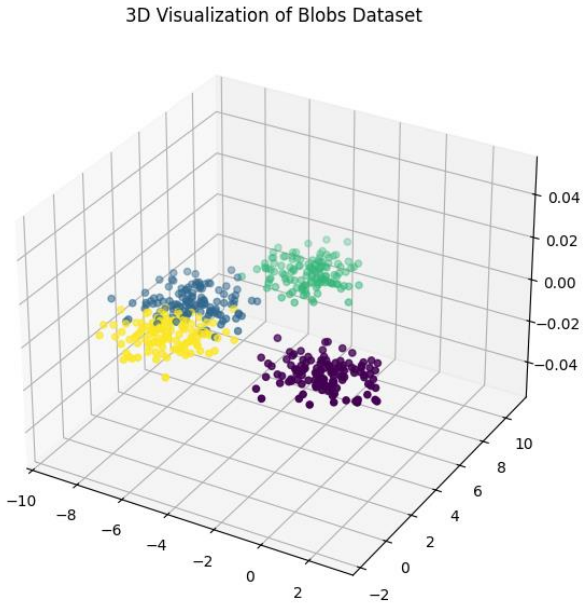
II. Clustering sur données synthétiques (Blobs)

Objectif : Comparer les performances de trois algorithmes sur des données bien séparées

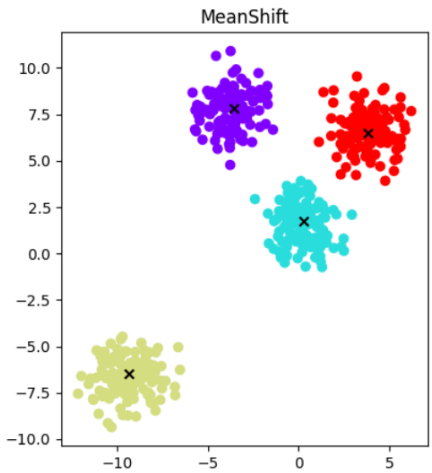
2.1 Génération des données

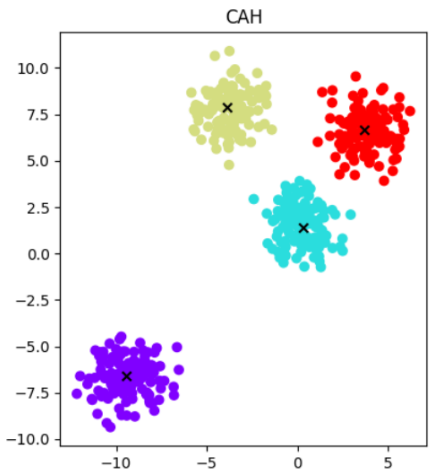
```
from sklearn.datasets import make_blobs
nbr_clusters=4
std_clusters=1
x,y,centers = make_blobs(n_samples=500, n_features=2, centers=nbr_clusters,
cluster_std=std_clusters, random_state=101,return_centers=True)
```

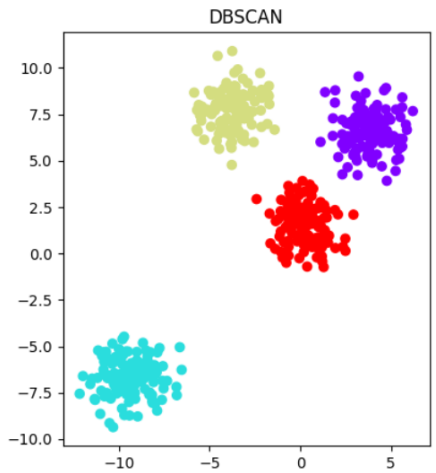


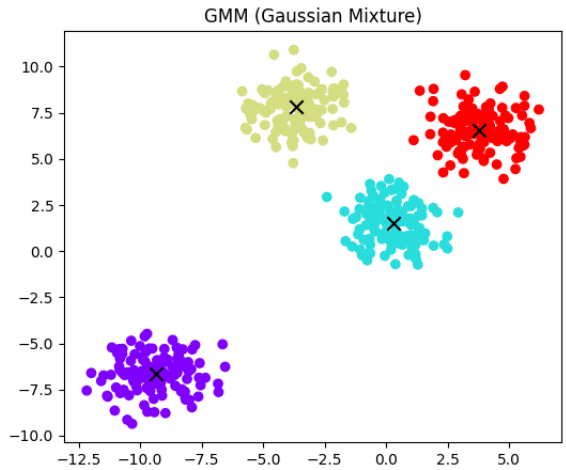


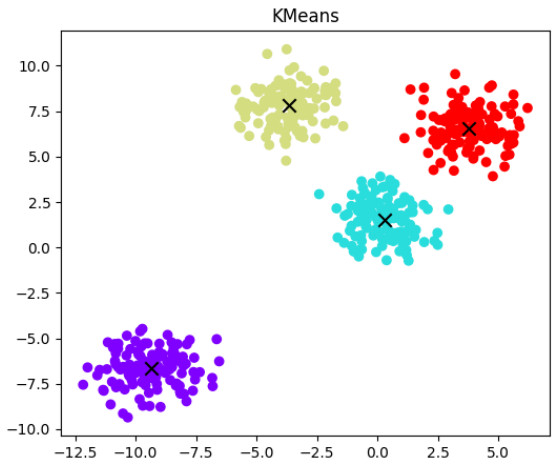
2.2 Application des algorithmes











2.3 Questions

1. Visualisez les résultats des algorithmes. Que constatez-vous ?
2. Comparez le nombre de clusters détectés par chaque algorithme. Pourquoi MeanShift peut-il trouver un nombre différent de clusters ?
3. Calculez et comparez les scores de silhouette et l'ARS pour chaque algorithme.
4. Quel algorithme performe le mieux sur ce type de données ? Justifiez.

À retenir : Le paramètre **bandwidth** de MeanShift et **eps** de DBSCAN sont cruciaux pour les performances

III. Segmentation d'image simple

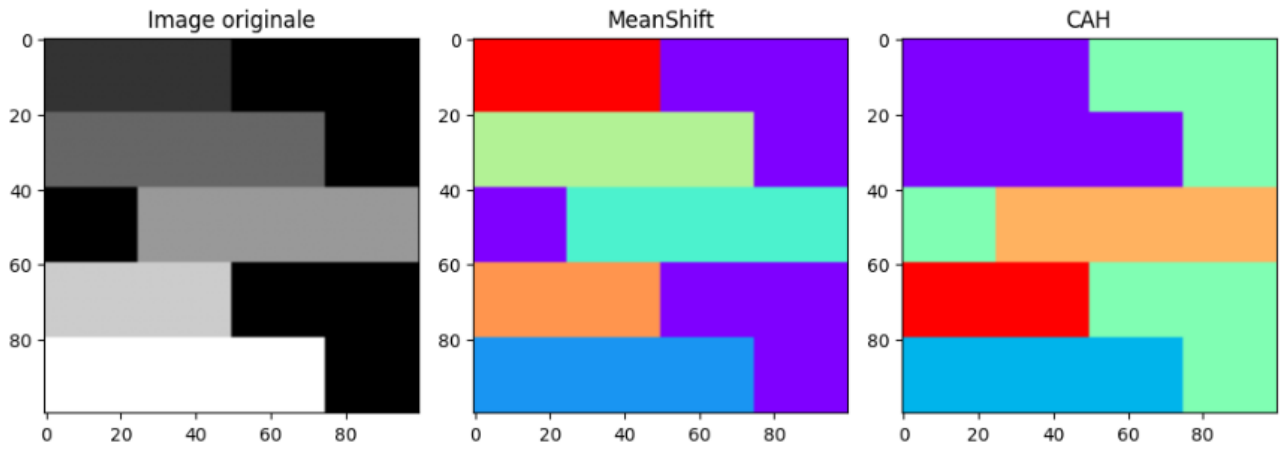
Objectif : Appliquer le clustering pour segmenter une image synthétique en régions

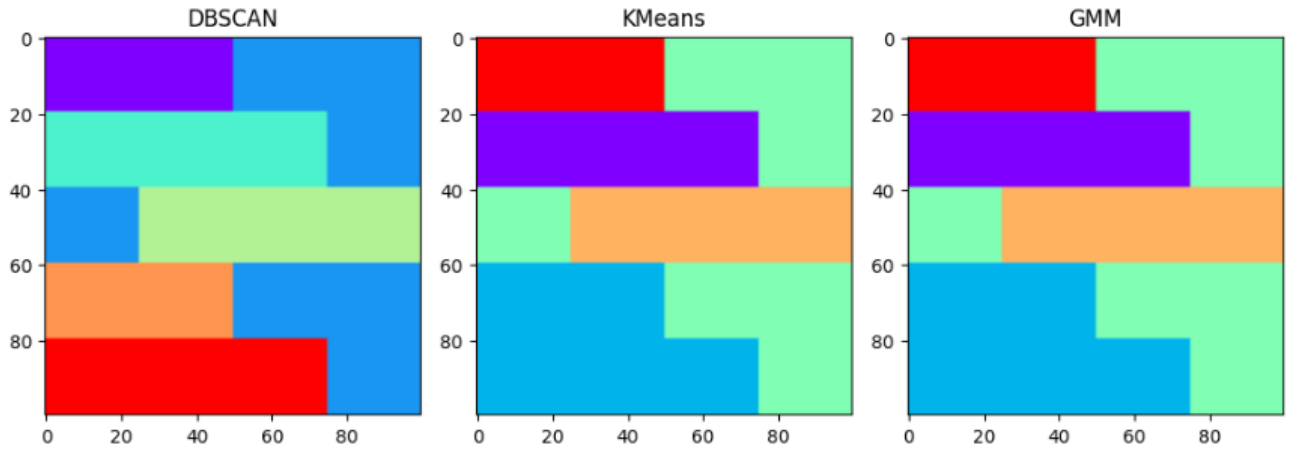
3.1 Création de l'image : Création d'une image 100x100 avec 5 niveaux de gris

```
image = np.zeros((100, 100))
image[0:20, 0:50] = 50
image[20:40, 0:75] = 100
image[40:60, 25:100] = 150
image[60:80, 0:50] = 200
image[80:100, 0:75] = 250

data = image.reshape(-1, 1)
```

3.2 Application des algorithmes





3.3 Questions

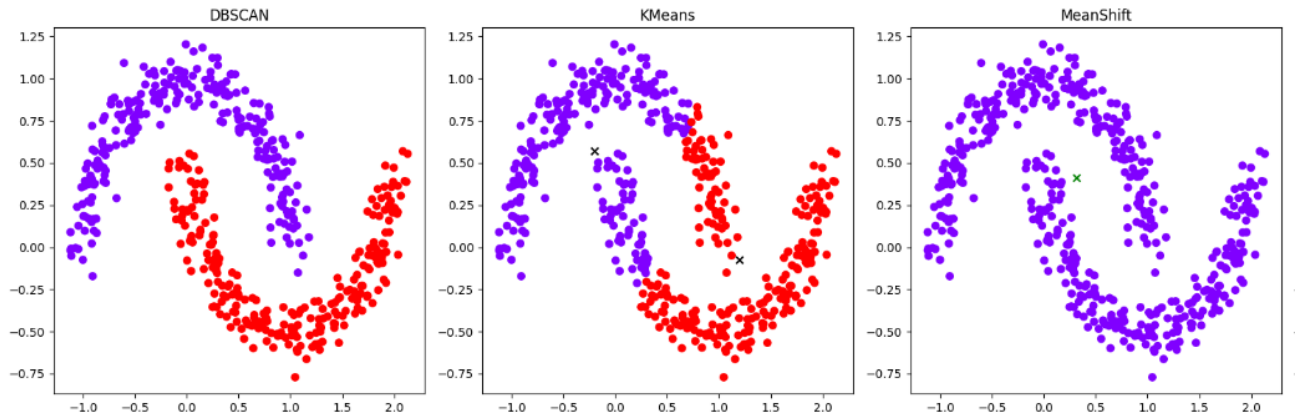
1. Affichez l'image originale et les résultats de segmentation des algorithmes.
2. Quel algorithme segmente le mieux les 5 régions distinctes ? Pourquoi ?
3. Que se passe-t-il si vous modifiez le paramètre **eps** de DBSCAN ?

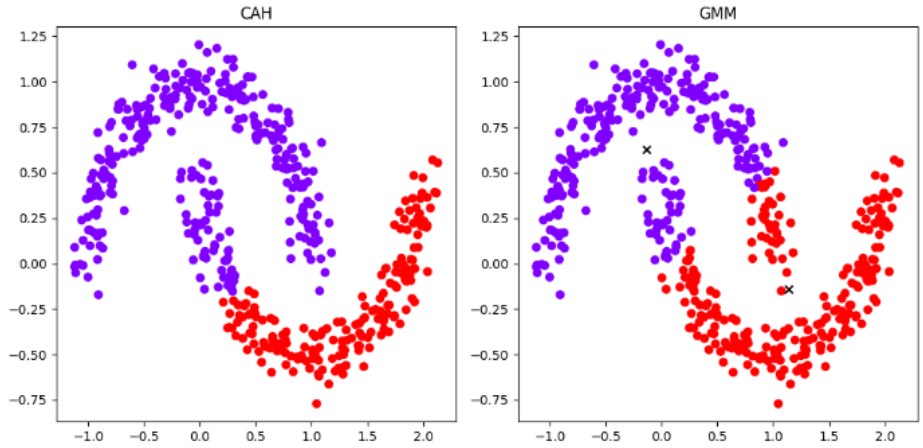
IV. Formes complexes (Moons et Circles)

Objectif : Tester les algorithmes sur des distributions non convexes

4.1 Dataset "Moons"

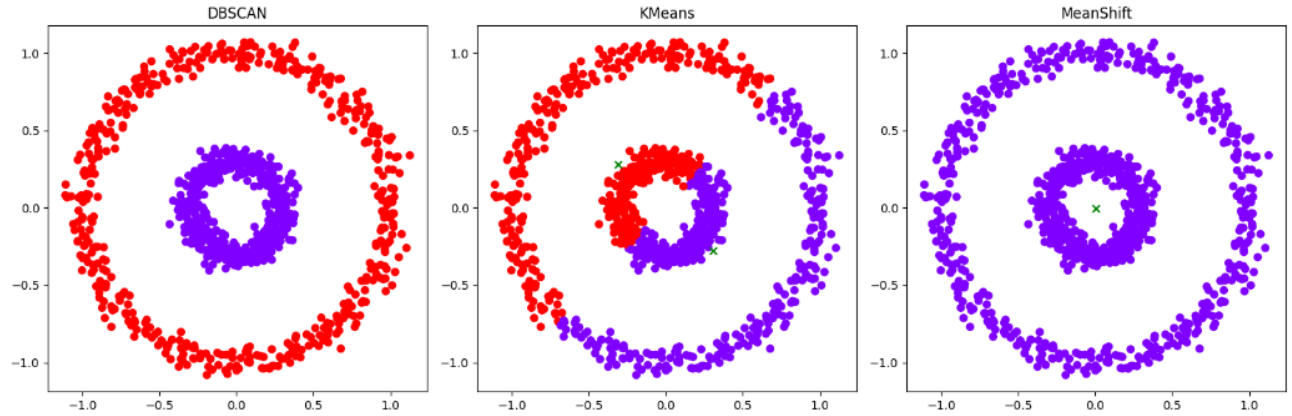
```
from sklearn.datasets import make_moons
x, y = make_moons(n_samples=500, noise=0.09, random_state=0)
fig, ax = plt.subplots(2,4, figsize=(20, 20))
```

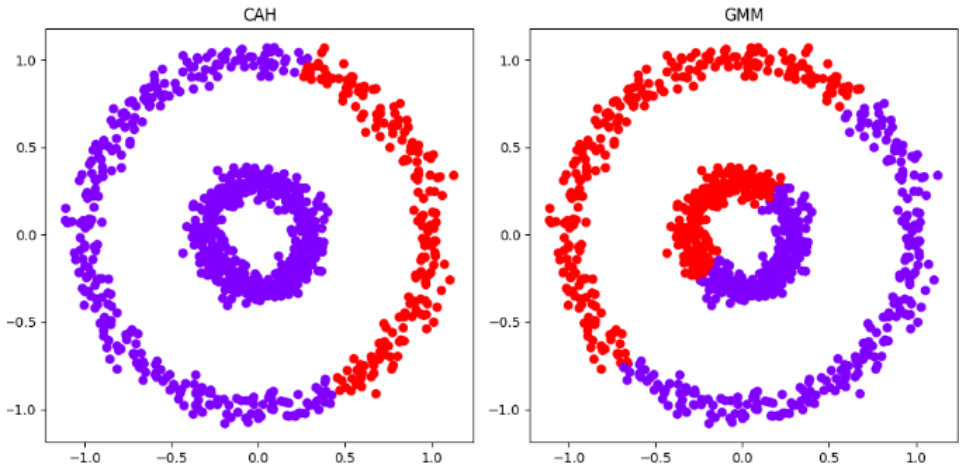




4.2 Dataset "Circles"

```
from sklearn.datasets import make_circles
x, y = make_circles(n_samples=1_000, factor=0.3, noise=0.05, random_state=0)
```





4.3 Questions

1. Visualisez les résultats pour chaque algorithme sur les deux datasets.
2. Quel algorithme échoue sur les Moons ? Pourquoi ?

3. Même question pour les Circles.
4. Expliquez pourquoi DBSCAN performe mieux sur ces formes complexes.

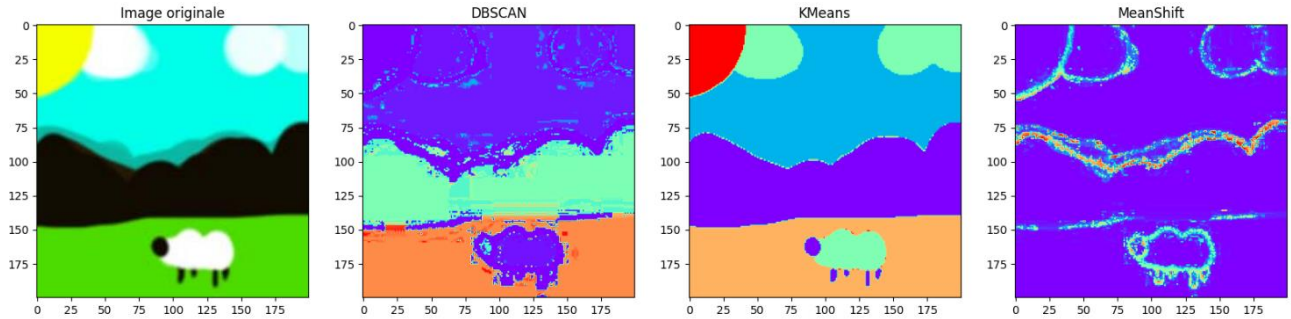
V. Segmentation d'image couleur

Objectif : Appliquer le clustering sur une image réelle pour la segmentation

5.1 Chargement et préparation

```
import cv2 as cv
img= cv.imread('/path.jpg')
img= cv.cvtColor(img, cv.COLOR_BGR2RGB)
data = img.reshape(-1, 3)
```

5.2 Application des algorithmes



5.3 Questions

1. Comparez visuellement les segmentations obtenues.
2. Quel algorithme donne la segmentation la plus cohérente ?
3. Pourquoi DBSCAN est-il moins adapté aux images couleur ?
4. Testez différentes valeurs de **bandwidth** pour MeanShift. Quel impact ?

© 2025 - Machine Learning & Intelligence Artificielle